



Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática de Albacete

Trabajo Fin de Grado
Grado en Ingeniería Informática
Computación

**Especificación de formato y desarrollo de
herramientas para el procesamiento de
curriculum vitae en el ámbito académico
universitario**

Carlos Martínez Jaén

Julio, 2026

TRABAJO FIN DE GRADO
Grado en Ingeniería Informática
Computación

**Especificación de formato y desarrollo de
herramientas para el procesamiento de
curriculum vitae en el ámbito académico
universitario**

Autor: Carlos Martínez Jaén

Director: Luis De La Ossa Jiménez

Codirector: José Antonio Gámez Martín

Julio, 2026

*Aquí va la dedicatoria que cada cual
quiera escribir. El ancho se controla
manualmente*

Declaración de autoría

Yo, ... , con DNI ... , declaro que soy el único autor del trabajo fin de grado titulado “ ... ”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a ... de ... de 20 ...

Fdo.: Carlos Martínez Jaén

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice general

1	Introducción, motivación y objetivos.....	1
1.1	Contexto.....	1
1.2	Motivación.....	2
1.3	Objetivos	3
1.4	Alcance del proyecto	5
1.5	Competencias desarrolladas.....	6
1.6	Contribuciones.....	7
1.7	Estructura del documento	8
2	Antecedentes y estado del arte	11
2.1	Representación de información curricular compleja	11
2.2	Interoperabilidad curricular y sistemas de información de investigación	12
2.3	Formatos de serialización y validación	14
2.3.1	<i>XML</i> y <i>XSD</i>	14
2.3.2	<i>JSON</i> y <i>JSON Schema</i>	15
2.3.3	<i>JSON-LD</i> y <i>YAML</i>	15
2.4	Modelado conceptual y generación de artefactos	17
2.5	Síntesis del estado del arte.....	18
3	Análisis del ecosistema CVN y propuesta de solución	21
3.1	El paquete oficial CVN como fuente del proyecto.....	21
3.2	El XML embebido en los documentos CVN.....	24
3.3	Limitaciones y dificultades técnicas detectadas.....	24
3.4	Propuesta de solución: la arquitectura Open CVN	27

3.5 Alcance y exclusiones	29
4 Metodología, herramientas y arquitectura general.....	31
4.1 Metodología de trabajo	31
4.2 Herramientas empleadas	32
4.3 Arquitectura general por capas	35
4.3.1 Fuente oficial CVN	35
4.3.2 Generación estructural y normalización.....	35
4.3.3 Dominio, modelo conceptual y formato	36
4.3.4 Herramienta local.....	37
4.4 Contratos entre capas y razonamiento arquitectónico	38
Referencia bibliográfica	41

Índice de figuras

3.1	Separación entre el paquete oficial CVN y la arquitectura Open CVN. . . .	27
3.2	Vista UML compacta del esquema conceptual de Open CVN, con el número de conceptos identificados por área curricular.	28
4.1	Arquitectura general de Open CVN organizada por capas, con el módulo del repositorio responsable de cada bloque.	35

Índice de tablas

1.1	Relación entre objetivos específicos y capítulos del documento.	5
2.1	Comparativa de modelos de interoperabilidad curricular relacionados con Open CVN.	13
2.2	Comparativa de formatos de serialización y validación relevantes para Open CVN.	17
3.1	Fuentes del paquete oficial CVN analizadas y su uso en el proyecto.	23
3.2	Limitaciones detectadas en el paquete oficial CVN y respuesta de diseño de Open CVN.	26
4.1	Herramientas empleadas en el proyecto y justificación de su uso.	34
4.2	Módulos del repositorio y responsabilidades dentro de la arquitectura Open CVN.	37

Índice de algoritmos

Índice de listados de código

1. Introducción, motivación y objetivos

El presente capítulo introduce el contexto general en el que se enmarca este Trabajo Fin de Grado, describe la motivación que justifica su desarrollo y delimita los objetivos que guían la solución propuesta. Para ello, en primer lugar se analiza el papel del Currículum Vítae Normalizado (CVN) en la gestión de información académica e investigadora en España. A continuación, se expone la necesidad de disponer de una representación de bajo nivel que facilite el procesamiento automático de los datos curriculares. Finalmente, se establecen el alcance del proyecto, las competencias de Computación trabajadas, las principales contribuciones realizadas y la estructura seguida por el resto del documento.

1.1. Contexto

La actividad académica e investigadora produce una cantidad considerable de información curricular de naturaleza heterogénea. En un mismo currículum pueden aparecer datos relativos a formación, experiencia profesional, publicaciones, proyectos de investigación, contratos, estancias, dirección de trabajos, transferencia, actividad docente y otros méritos que responden a estructuras y criterios de descripción diferentes. Esta información no se utiliza únicamente como documento de presentación personal, sino también como entrada para procesos de evaluación, convocatorias públicas, solicitudes de ayudas, acreditaciones, portales institucionales y sistemas internos de gestión de la investigación. En consecuencia, el valor de un currículum no depende solo de su apariencia final, sino también de la capacidad de los sistemas informáticos para interpretar, validar, reutilizar y transformar los datos que contiene.

En el ámbito español, el Currículum Vítae Normalizado (CVN) constituye una norma de referencia para la presentación de información curricular de investigadores. La Fundación Española para la Ciencia y la Tecnología (FECYT) lo define como una norma estándar que

establece un formato común de presentación de los datos curriculares y facilita la interoperabilidad con bases de datos institucionales [FECYT, 2026]. Esta función normalizadora resulta especialmente relevante en un entorno en el que una misma trayectoria académica puede tener que presentarse ante organismos, plataformas y convocatorias con requisitos distintos. Por ello, CVN representa un punto de partida adecuado para estudiar cómo puede formalizarse la información curricular de forma computable y reutilizable.

No obstante, la existencia de un formato normalizado no elimina por sí misma el coste asociado a la elaboración, el mantenimiento y la adaptación de los currículos. A pesar de disponer de una referencia común, los profesores e investigadores universitarios continúan dedicando un tiempo significativo a actualizar méritos, preparar versiones específicas, adaptar contenidos a formatos concretos o trasladar información entre herramientas. Esta situación se ve agravada cuando el mismo historial debe reutilizarse en contextos diferentes, ya que pequeñas variaciones en la estructura, el nivel de detalle o el formato de salida pueden obligar a realizar tareas manuales repetitivas. Desde esta perspectiva, resulta necesario avanzar hacia herramientas abiertas que automaticen parte del proceso y que permitan gestionar los currículos de manera más sistemática.

El tratamiento computacional de CVN, sin embargo, no puede reducirse a la generación de un documento final ni a la lectura de un único fichero XML. La base del trabajo es la norma CVN y el ecosistema de artefactos que la acompaña; dentro de dicho ecosistema, XML actúa como mecanismo de representación e intercambio de datos. En particular, el paquete oficial analizado en este trabajo incluye esquemas XML/XSD, manuales estructurados, modelos auxiliares y tablas de referencia que describen la estructura de los datos, sus códigos internos y su organización jerárquica. Estos elementos aportan la base formal necesaria para la interoperabilidad, pero también introducen una complejidad que dificulta su uso directo como modelo interno de herramientas abiertas, ligeras y reproducibles. En conjunto, el problema abordado consiste en separar el significado de la información curricular de la forma concreta en que la norma la representa y la serializa en sus artefactos técnicos.

1.2. Motivación

La motivación principal de este proyecto surge de la necesidad de definir una representación de bajo nivel para currículos académicos e investigadores basada en la norma CVN, pero orientada al procesamiento automático de datos. Dicha representación debe permitir que la información curricular pueda almacenarse, validarse, transformarse y exportarse sin depender exclusivamente de la representación XML empleada por CVN ni de la herramienta oficial

asociada. Esta necesidad está directamente relacionada con la posibilidad de desarrollar herramientas abiertas para la creación, actualización y adaptación de currículos, tal y como se plantea en la descripción oficial del Trabajo Fin de Grado.

Desde el punto de vista técnico, el uso directo de los artefactos oficiales de CVN como modelo interno presenta varias dificultades. Los esquemas XSD proporcionan una descripción estructural precisa de la representación XML, pero no siempre expresan de forma explícita la semántica curricular necesaria para construir aplicaciones de alto nivel. Además, parte de la información relevante se encuentra distribuida entre manuales, modelos auxiliares y tablas de referencia, lo que obliga a combinar varias fuentes para interpretar correctamente los campos, códigos y relaciones del dominio. Esta dispersión no invalida la norma CVN ni sus artefactos oficiales, que cumplen una función esencial de interoperabilidad, pero sí condiciona su reutilización como base directa para herramientas de gestión, validación o exportación.

El presente Trabajo Fin de Grado aborda este problema desde una perspectiva de Computación. La cuestión central no consiste únicamente en leer o escribir ficheros, sino en analizar fuentes formales heterogéneas, extraer evidencia estructural y semántica, normalizar metadatos, preservar trazabilidad y generar modelos capaces de representar información curricular compleja. De este modo, el proyecto se sitúa en el ámbito de la formalización y representación computable del conocimiento, con especial atención a la separación entre formato de serialización, modelo conceptual y artefactos de validación.

Como respuesta a esta situación, Open CVN propone una base técnica abierta, extensible y potencialmente colaborativa para representar currículos académicos tomando CVN como referencia. La solución no pretende sustituir la norma oficial, sino construir sobre ella una capa intermedia que facilite el desarrollo de herramientas posteriores. Esta capa debe permitir trabajar con un formato final basado en JSON, adecuado tanto para almacenamiento en archivos de texto como para su integración en bases de datos documentales o aplicaciones de procesamiento. De este modo, la norma CVN y sus artefactos oficiales se mantienen como fuentes de trazabilidad e interoperabilidad, mientras que el formato Open CVN se orienta a facilitar la gestión automatizada y la evolución controlada de los datos curriculares sin quedar limitado a la representación XML ni a la herramienta oficial existente.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Grado es *diseñar e implementar una arquitectura computacional abierta para representar, validar, transformar, almacenar y exportar currículos académicos e investigadores tomando CVN como norma y ecosistema de referen-*

cia, pero evitando que su representación XML o la herramienta oficial asociada condicionen por completo el modelo interno del sistema, con el fin de facilitar el desarrollo de herramientas abiertas para la gestión automatizada de currículos.

Para alcanzar esta meta global se han definido los siguientes objetivos específicos:

1. **Analizar el ecosistema CVN y sus artefactos oficiales.** Este objetivo comprende el estudio de los esquemas XML/XSD, manuales estructurados, modelos auxiliares y tablas de referencia que forman parte del paquete empleado como fuente del proyecto.
2. **Estudiar alternativas de representación y validación de datos curriculares.** Se consideran modelos conceptuales, formatos de serialización y herramientas de generación de modelos con el fin de seleccionar una estrategia adecuada para información curricular compleja.
3. **Generar una capa estructural reproducible a partir de los XSD oficiales.** La generación automática permite obtener artefactos consistentes con la representación XML definida dentro del ecosistema CVN y, al mismo tiempo, separar el código generado de la lógica mantenida manualmente.
4. **Normalizar metadatos procedentes de las fuentes oficiales.** Este objetivo busca unificar la información funcional y técnica asociada a los códigos CVN, reduciendo la dispersión existente entre documentos auxiliares.
5. **Definir reglas semánticas conservadoras y trazables.** Las reglas deben permitir interpretar campos, tipos, tablas de referencia y relaciones sin perder la conexión con las fuentes originales de las que se derivan.
6. **Generar modelos de dominio y artefactos conceptuales.** Estos modelos deben representar la información curricular de forma más próxima al dominio académico que a la estructura concreta de la serialización XML.
7. **Definir un formato Open CVN JSON.** El formato propuesto debe ser validable, extensible y adecuado para intercambio, almacenamiento, colaboración y transformación posterior.
8. **Implementar mecanismos de parseo, validación, almacenamiento y exportación.** Estos mecanismos permiten demostrar que la representación definida puede emplearse en flujos reales de gestión curricular y generación de artefactos de salida.
9. **Diseñar mecanismos de importación deterministas y asistidos por modelos de lenguaje.** El objetivo consiste en incorporar a Open CVN currículos ya existentes en CVN

XML o en CVN PDF, priorizando siempre una vía de extracción determinista y reservando el uso de modelos de lenguaje de gran tamaño (del inglés, *Large Language Models*, en adelante LLM) como alternativa opcional para los casos en que la vía determinista no resulta suficiente. Esta vía asistida debe activarse de forma explícita, mantenerse trazable y quedar sujeta a validación posterior frente al formato Open CVN antes de aceptarse como resultado válido.

10. **Verificar el sistema mediante pruebas reproducibles.** La evaluación se apoya en pruebas automatizadas, comandos documentados y flujos de validación extremo a extremo, con el fin de establecer las garantías reales del sistema desarrollado.

La Tabla 1.1 relaciona estos objetivos con los capítulos principales en los que se desarrollan.

Objetivo	Descripción resumida	Capítulos
OE1	Análisis del ecosistema CVN y sus artefactos oficiales	2, 3
OE2	Estudio de alternativas de representación, modelado y validación	2
OE3	Generación estructural reproducible desde XSD	4, 5
OE4	Normalización de metadatos CVN	5
OE5	Definición de reglas semánticas trazables	3, 5
OE6	Generación de modelos de dominio y artefactos conceptuales	5
OE7	Definición del formato Open CVN JSON	6
OE8	Parser, validador, almacenamiento y exportación	6
OE9	Importación determinista y asistida por LLM	6
OE10	Verificación automatizada y flujos reproducibles	7

Tabla 1.1: Relación entre objetivos específicos y capítulos del documento.

1.4. Alcance del proyecto

El alcance del proyecto se centra en la construcción de una base computacional para trabajar con información curricular estructurada. El trabajo incluye el análisis del paquete oficial CVN empleado como fuente, la generación de bindings estructurales, la normalización de metadatos, la definición de reglas semánticas, la generación de modelos de dominio,

la especificación de un formato JSON propio y la implementación de herramientas de validación, importación, almacenamiento y exportación. Estas tareas se han organizado de forma que cada capa conserve trazabilidad hacia las fuentes originales y pueda verificarse mediante procedimientos reproducibles.

Dentro del alcance se incluye también la importación de currículos ya existentes hacia Open CVN a partir de CVN XML y de CVN PDF. Esta importación se resuelve en primer lugar mediante extracción determinista y solo recurre, de forma opcional y explícitamente autorizada, a modelos de lenguaje (LLM) cuando la vía determinista no permite recuperar información suficiente. El resultado de esta vía asistida no se presenta como una validación semántica completa del currículo importado, sino como una propuesta que debe superar la validación del formato Open CVN y quedar disponible para su revisión antes de considerarse definitiva.

Queda fuera del alcance garantizar una conversión completa y sin pérdida de todos los currículos CVN existentes, ya que dicha garantía requeriría cubrir la totalidad de variantes, casos particulares y restricciones prácticas del ecosistema oficial. Tampoco se pretende sustituir la norma CVN ni la infraestructura proporcionada por FECYT. La propuesta debe entenderse como una capa abierta y reproducible que facilita el tratamiento de datos curriculares cuando se necesita validación, transformación, almacenamiento local o exportación a otros formatos.

La herramienta local desarrollada dentro del proyecto cumple una función demostrativa y operativa. Su finalidad es comprobar que el formato propuesto y las capas de validación pueden utilizarse en flujos de importación, almacenamiento, versionado y exportación. No obstante, la contribución principal del TFG no reside en la interfaz de dicha herramienta, sino en la arquitectura de representación, normalización y transformación que permite construirla.

1.5. Competencias desarrolladas

El desarrollo del proyecto se alinea con competencias propias de la intensificación de Computación. En particular, de acuerdo con la descripción oficial del trabajo, se abordan las siguientes:

- **[CM1]** Capacidad para tener un conocimiento profundo de los principios fundamentales y modelos de la computación y saberlos aplicar para interpretar, seleccionar, valorar, modelar y crear nuevos conceptos, teorías, usos y desarrollos tecnológicos relacionados con la informática.

- **[CM2]** Capacidad para conocer los fundamentos teóricos de los lenguajes de programación y las técnicas de procesamiento léxico, sintáctico y semántico asociadas, y saber aplicarlas para la creación, diseño y procesamiento de lenguajes.
- **[CM5]** Capacidad para adquirir, obtener, formalizar y representar el conocimiento humano en una forma computable para la resolución de problemas mediante un sistema informático en cualquier ámbito de aplicación, particularmente los relacionados con aspectos de computación, percepción y actuación en ambientes o entornos inteligentes.
- **[CM6]** Capacidad para desarrollar y evaluar sistemas interactivos y de presentación de información compleja y su aplicación a la resolución de problemas de diseño de interacción persona-computadora.

Estas competencias se reflejan en el análisis de estructuras oficiales complejas, la formalización de información curricular en modelos computables, la generación automática de artefactos, la definición de reglas de validación, la construcción de una herramienta de gestión y la evaluación técnica del sistema mediante pruebas reproducibles. Su grado de cumplimiento se retomará en el capítulo de conclusiones, una vez presentados el desarrollo y los resultados obtenidos.

1.6. Contribuciones

Las principales contribuciones del trabajo pueden resumirse en los siguientes elementos:

- **Análisis computacional del ecosistema CVN:** se identifican las fuentes estructurales, funcionales y auxiliares necesarias para interpretar la información curricular representada en CVN, así como las restricciones que condicionan su uso automatizado.
- **Arquitectura reproducible por capas:** se separan los bindings generados, la normalización de metadatos, la política semántica, los modelos de dominio, el modelo conceptual, el formato JSON, la validación y la herramienta local, evitando que una única representación concentre todas las responsabilidades del sistema.
- **Trazabilidad entre CVN y Open CVN:** se conserva evidencia sobre códigos, fuentes, rutas y decisiones semánticas, con el fin de que la transformación hacia el nuevo formato mantenga una relación verificable con el estándar original.
- **Formato Open CVN JSON:** se define una representación abierta, validable y extensible orientada al intercambio, almacenamiento y exportación de datos curriculares.

-
- **Herramientas de validación y uso:** se implementan parsers, validadores, almacenamiento local, versionado curricular y exportación a otros formatos, lo que permite comprobar la utilidad práctica de la representación definida.
 - **Importación determinista y asistida por LLM:** se implementan vías de importación desde CVN XML y CVN PDF hacia Open CVN, priorizando la extracción determinista y reservando el uso de modelos de lenguaje de gran tamaño como alternativa opcional, trazable y sujeta a validación posterior para los casos en que la extracción determinista resulta insuficiente.
 - **Verificación automatizada:** se incorporan pruebas y flujos reproducibles para evaluar la consistencia de los artefactos generados y el comportamiento de las capas principales del sistema.

En conjunto, estas contribuciones no deben entenderse como una sustitución del ecosistema CVN, sino como una propuesta complementaria que facilita su tratamiento computacional en contextos donde resultan necesarias la validación formal, la transformación entre formatos y la gestión automatizada de currículos.

1.7. Estructura del documento

El resto del documento se organiza de forma progresiva, desde los antecedentes y el análisis del dominio hasta la implementación, evaluación y discusión de la solución desarrollada:

- **Capítulo 2: Antecedentes y estado del arte.** Presenta el contexto técnico y conceptual necesario para comprender el problema, incluyendo CVN, la interoperabilidad curricular, los modelos relacionados, los formatos de serialización y las alternativas de modelado.
- **Capítulo 3: Análisis del ecosistema CVN y propuesta de solución.** Describe las fuentes oficiales utilizadas, sus limitaciones y la propuesta de arquitectura Open CVN derivada de dicho análisis.
- **Capítulo 4: Metodología, herramientas y arquitectura general.** Explica el proceso técnico seguido, las herramientas empleadas y la organización por capas del sistema.
- **Capítulo 5: Implementación del *pipeline* de generación y normalización.** Detalla la generación estructural, la normalización de metadatos, la política semántica y la generación de modelos y artefactos conceptuales.

- **Capítulo 6: Formato Open CVN, validación y herramienta de gestión.** Presenta el formato JSON propuesto, los parsers, validadores, las vías de importación determinista y asistida por LLM, los mecanismos de almacenamiento local y los flujos de exportación.
- **Capítulo 7: Evaluación, resultados y discusión.** Describe la estrategia de verificación, los resultados obtenidos y las garantías reales que ofrece el sistema desarrollado.
- **Capítulo 8: Conclusiones, competencias y trabajo futuro.** Resume el grado de cumplimiento de los objetivos, relaciona el trabajo con las competencias desarrolladas, recoge las limitaciones identificadas y plantea líneas futuras.

2. Antecedentes y estado del arte

El presente capítulo recoge los antecedentes técnicos que sustentan el trabajo: interoperabilidad de información investigadora, formatos de serialización, mecanismos de validación y técnicas de modelado conceptual. Su finalidad es comparar alternativas y justificar las decisiones tecnológicas que se utilizarán en los capítulos posteriores, sin desarrollar todavía el análisis específico del dominio CVN. Ese análisis se reserva para el capítulo siguiente, donde se estudiarán sus artefactos oficiales, sus limitaciones y la propuesta Open CVN construida a partir de ellos.

2.1. Representación de información curricular compleja

La información curricular académica e investigadora constituye un dominio de datos heterogéneo, con entidades, relaciones, restricciones y niveles de detalle que no encajan bien en una representación plana. Desde una perspectiva computacional, esta circunstancia obliga a diferenciar el modelo conceptual del dominio, la serialización utilizada para intercambiar datos y las herramientas que permiten procesarlos. Esta separación será una idea recurrente en el resto del documento, porque permite razonar sobre el significado de los datos sin confundirlo con el formato concreto en el que se almacenan o transmiten.

En España, CVN proporciona una referencia normalizada para la presentación de información curricular de investigadores y favorece la interoperabilidad con bases de datos institucionales [FECYT, 2026]. En este capítulo se menciona únicamente como punto de conexión con el dominio de aplicación. La descripción de sus fuentes, de su representación técnica y de sus limitaciones se reserva para el capítulo siguiente, con el fin de no anticipar el análisis que corresponde a la propuesta de solución.

2.2. Interoperabilidad curricular y sistemas de información de investigación

La gestión de información curricular no es un problema exclusivo de CVN. En el ámbito académico internacional existen diferentes iniciativas orientadas a mejorar la interoperabilidad entre sistemas de información de investigación. Estos sistemas, conocidos habitualmente como *Current Research Information Systems* (CRIS), integran datos sobre investigadores, organizaciones, proyectos, publicaciones, financiación, resultados científicos y otros elementos relevantes para la actividad investigadora.

La interoperabilidad en este dominio no depende solo de compartir documentos, sino también de poder identificar de forma estable a las personas, organizaciones, resultados y relaciones descritas. ORCID, por ejemplo, proporciona identificadores persistentes para investigadores y mecanismos de conexión entre dichos identificadores, contribuciones y afiliaciones [ORCID, 2026]. En una línea complementaria, RO-Crate propone un enfoque ligero para empaquetar datos de investigación junto con sus metadatos [RO-Crate Community, 2024]. Estas iniciativas no resuelven por sí mismas la representación completa de un currículum CVN, pero muestran la importancia de combinar datos estructurados, metadatos explícitos e identificadores reutilizables.

Uno de los modelos más representativos en este ámbito es CERIF, mantenido por euroCRIS. CERIF se presenta como un formato europeo para la gestión, intercambio y almacenamiento de información de investigación, con especial atención a la interoperabilidad entre sistemas CRIS [euroCRIS, 2026]. Su principal ventaja reside en la riqueza de su modelo, que permite representar entidades, relaciones y dimensiones temporales con un alto grado de precisión. Esta expresividad resulta adecuada para infraestructuras institucionales complejas, aunque también introduce un nivel de abstracción y complejidad que puede ser excesivo cuando el objetivo es definir una representación ligera y directamente procesable de currículos.

Otra aproximación relevante es VIVO, una plataforma orientada a conectar, compartir y descubrir información sobre investigación mediante tecnologías de Web Semántica [VIVO Project, 2026]. Frente a modelos más tabulares o documentales, VIVO se apoya en grafos y ontologías para describir investigadores, publicaciones, organizaciones y relaciones académicas. Esta orientación facilita la publicación de datos enlazados y la integración con vocabularios semánticos, pero exige una infraestructura conceptual y tecnológica más compleja que la requerida por una herramienta local de gestión curricular.

En el contexto español destaca también el proyecto HERCULES y, dentro de él, la

Research Ontology for Hercules (ROH). La red de ontologías HERCULES persigue la representación semántica de información de investigación en el sistema universitario español [Universidad de Murcia, 2026, Proyecto HERCULES, 2026]. Además, parte de su ecosistema se ha relacionado con CVN mediante código y artefactos públicos orientados a la importación de información curricular [Deusto HERCULES, 2026b, Deusto HERCULES, 2026a]. Esta proximidad al dominio español convierte a ROH en una referencia relevante, aunque su orientación ontológica y su dependencia de tecnologías de Web Semántica hacen que no sea la opción más sencilla para el alcance de este trabajo.

La Tabla 2.1 resume estos tres enfoques atendiendo al tipo de representación empleada, su principal fortaleza y la limitación que presenta cada uno frente al objetivo de Open CVN.

Modelo	Enfoque de representación	Fortaleza principal	Limitación frente a Open CVN
CERIF	Modelo europeo orientado a entidades y relaciones para sistemas CRIS	Alta expresividad para relaciones institucionales y dimensiones temporales	Complejidad y coste de adopción elevados para una representación ligera
VIVO	Grafos y ontologías de Web Semántica	Publicación de datos enlazados e integración con vocabularios semánticos	Requiere infraestructura RDF/ontológica no necesaria para el alcance del TFG
ROH	Ontología del ámbito universitario e investigador español (proyecto HERCULES)	Proximidad al dominio CVN español y artefactos de importación ya existentes	Orientación ontológica y dependencia de tecnologías de Web Semántica

Tabla 2.1: Comparativa de modelos de interoperabilidad curricular relacionados con Open CVN.

En conjunto, CERIF, VIVO y ROH muestran que la información investigadora puede modelarse con niveles elevados de riqueza semántica. No obstante, también ponen de manifiesto una tensión habitual entre expresividad e implementabilidad. Cuanto más completo y semánticamente rico es un modelo, mayor suele ser el coste de adopción, mantenimiento, consulta y validación. Para Open CVN, esta observación resulta determinante: el objetivo no es competir con infraestructuras CRIS ni con ontologías de investigación completas, sino definir una representación curricular abierta, validable y suficientemente expresiva para el ámbito del TFG.

2.3. Formatos de serialización y validación

La elección de un formato de serialización condiciona la forma en que los datos se almacenan, intercambian y procesan. Sin embargo, un formato de serialización no equivale por sí mismo a un modelo conceptual. Un mismo dominio puede representarse en XML, JSON, YAML u otros formatos, siempre que exista una definición clara de las entidades, atributos, relaciones y restricciones que deben preservarse. Por ello, en este trabajo los formatos se analizan como mecanismos técnicos, no como la única fuente de significado del sistema.

2.3.1. XML y XSD

Extensible Markup Language (XML) es un lenguaje de marcado definido por el World Wide Web Consortium y ampliamente utilizado para representar documentos y datos estructurados. La recomendación XML 1.0 lo define como un perfil de SGML diseñado para facilitar el intercambio y procesamiento de documentos en la Web, con atención explícita a la interoperabilidad y a la facilidad de implementación de procesadores [World Wide Web Consortium, 2008]. XSD se define, a su vez, como un lenguaje para describir la estructura y restringir el contenido de documentos XML, aunque la propia especificación reconoce que algunas aplicaciones pueden necesitar validaciones adicionales no expresables únicamente mediante el esquema [World Wide Web Consortium, 2012].

La principal ventaja de XML en este contexto es su capacidad para representar estructuras jerárquicas complejas y asociarlas a esquemas de validación robustos. Esto permite comprobar si un documento respeta una estructura determinada y facilita la interoperabilidad entre sistemas que comparten la misma definición. Además, el uso de XSD proporciona una fuente formal a partir de la cual pueden generarse artefactos estructurales, como clases de datos o bindings de acceso programático.

No obstante, XML también presenta limitaciones cuando se utiliza como base directa de una herramienta de gestión. Su sintaxis es más verbosa que la de formatos más ligeros, el procesamiento suele requerir bibliotecas específicas y la estructura del documento no siempre coincide con el modelo conceptual más adecuado para una aplicación. Por tanto, XML puede ser una fuente técnica fundamental en sistemas de intercambio estructurado, pero no constituye por sí solo el modelo completo del dominio representado.

2.3.2. JSON y JSON Schema

JavaScript Object Notation (JSON) es un formato ligero de intercambio de datos basado en estructuras de pares clave-valor y listas ordenadas [JSON.org, 2026]. Su sencillez sintáctica, su integración directa con lenguajes de programación y su uso extendido en servicios web lo convierten en una alternativa adecuada para representaciones orientadas al procesamiento automático. En Python, por ejemplo, un documento JSON puede mapearse de forma natural a diccionarios, listas y tipos primitivos, lo que simplifica su manipulación en aplicaciones.

La principal dificultad de JSON es que, por sí solo, no define restricciones estructurales ni semánticas. Para resolver esta limitación se utiliza JSON Schema, un vocabulario que permite describir la estructura esperada de documentos JSON, incluyendo tipos, campos obligatorios, restricciones de valores y composición de objetos. La documentación de JSON Schema lo presenta como un lenguaje declarativo para definir estructura y restricciones de datos JSON, cuya validación requiere comprobar una instancia contra un esquema mediante herramientas específicas [JSON Schema, 2026]. De este modo, JSON puede combinarse con un mecanismo de validación formal similar en propósito a XSD, aunque con una sintaxis más cercana a los formatos habituales de intercambio de datos en aplicaciones modernas.

En el ecosistema Python, Pydantic complementa esta aproximación al permitir definir modelos mediante anotaciones de tipo, validar datos de entrada, serializar estructuras y emitir JSON Schema a partir de los modelos [Pydantic, 2026]. Esta combinación resulta especialmente adecuada para un proyecto que necesita mantener correspondencia entre un formato de intercambio, reglas de validación y objetos manipulables desde código. Al mismo tiempo, obliga a distinguir entre la validación sintáctica o estructural, que puede comprobarse de forma automática, y las reglas semánticas del dominio, que requieren análisis adicional de las fuentes CVN.

Para el contexto de Open CVN, JSON resulta especialmente adecuado como formato final de intercambio y almacenamiento porque permite construir documentos legibles, validables y compatibles con herramientas habituales de desarrollo. Esta elección no implica renunciar a una referencia normativa del dominio. Al contrario, el objetivo consiste en derivar una representación JSON trazable a partir de fuentes oficiales, preservando la relación entre los datos, sus reglas y su interpretación curricular.

2.3.3. JSON-LD y YAML

JSON for Linking Data (JSON-LD) extiende JSON para facilitar la representación de datos enlazados mediante contextos semánticos. La recomendación JSON-LD 1.1 lo de-

fine como una serialización basada en JSON para *Linked Data*, pensada para integrarse con sistemas que ya utilizan JSON y facilitar un camino de evolución hacia datos enlazados [World Wide Web Consortium, 2020]. Su principal aportación consiste en permitir que un documento JSON pueda interpretarse también como parte de un grafo de conocimiento, asociando claves del documento con identificadores semánticos. Esta característica lo convierte en una opción interesante cuando se busca compatibilidad con tecnologías RDF u ontologías.

Sin embargo, la incorporación de semántica enlazada también aumenta la complejidad del sistema. Para un proyecto cuyo objetivo principal es definir una representación curricular abierta, validable y manejable mediante herramientas Python, JSON-LD puede resultar más expresivo de lo necesario en una primera aproximación. Su interés se mantiene como posible línea de evolución futura, en especial si Open CVN necesitara integrarse con ontologías como ROH o con infraestructuras de datos enlazados.

YAML, por su parte, es un formato orientado a la legibilidad humana y empleado con frecuencia en configuración y definición de esquemas [YAML.org, 2026]. Su sintaxis compacta facilita la lectura manual, aunque puede introducir ambigüedades y requiere parsers específicos. En el contexto de este trabajo, YAML resulta relevante principalmente por su uso en herramientas de modelado como LinkML, pero no se adopta como formato final de intercambio curricular.

La Tabla 2.2 recoge, a modo de síntesis, la función principal, el mecanismo de validación asociado y el papel que cada uno de estos formatos desempeña dentro de Open CVN.

Formato	Función principal	Validación asociada	Papel en Open CVN
XML + XSD	Representación jerárquica de documentos y datos estructurados	Esquema XSD	Fuente estructural oficial de CVN; no se adopta como formato final de intercambio
JSON + JSON Schema	Intercambio ligero de datos orientado a aplicaciones	JSON Schema, con apoyo de Pydantic en Python	Formato final de intercambio, almacenamiento y validación de Open CVN
JSON-LD	Extensión de JSON para datos enlazados semánticamente	Contextos JSON-LD y vocabularios semánticos	Línea de evolución futura; no se adopta en la versión actual
YAML	Formato legible orientado a configuración y definición de esquemas	Depende de la herramienta que lo consume (por ejemplo, LinkML)	Referencia en herramientas de modelado; no se emplea como formato de intercambio final

Tabla 2.2: Comparativa de formatos de serialización y validación relevantes para Open CVN.

2.4. Modelado conceptual y generación de artefactos

La separación entre modelo conceptual y formato de serialización conduce a la necesidad de utilizar técnicas de modelado que permitan describir el dominio de forma independiente de una tecnología concreta. En este sentido, Unified Modeling Language (UML) constituye una referencia ampliamente utilizada para especificar, visualizar y documentar sistemas mediante clases, atributos, relaciones, jerarquías y multiplicidades [Object Management Group, 2026b]. Su utilidad en este trabajo reside en la posibilidad de representar conceptos curriculares y sus relaciones sin depender inicialmente de XML, JSON u otro formato de intercambio.

Por este motivo, UML se utilizará como notación principal para representar el esquema conceptual derivado del análisis del dominio. En particular, los diagramas de clases permiten expresar entidades curriculares, atributos, asociaciones y multiplicidades en un nivel intermedio entre la descripción normativa de CVN y los artefactos técnicos generados posteriormente. Esta representación no sustituye a los modelos ejecutables ni a los esquemas de validación, pero proporciona una vista comprensible y tecnológicamente independiente del esquema que se desea obtener.

Cuando el modelo requiere restricciones más precisas que las expresables mediante clases

y relaciones, Object Constraint Language (OCL) permite definir condiciones formales sobre modelos UML [Object Management Group, 2026a]. Estas restricciones pueden utilizarse para especificar reglas de validez, cardinalidad o consistencia que no siempre quedan reflejadas de forma natural en un diagrama estructural. Aunque este trabajo no se basa exclusivamente en UML/OCL como fuente de generación final, estos conceptos resultan relevantes para justificar la separación entre estructura, semántica y validación.

En esta línea, existen herramientas que permiten generar artefactos técnicos a partir de modelos. BESSER, por ejemplo, proporciona un generador de modelos Pydantic a partir de especificaciones de modelado [BESSER, 2026]. Este tipo de enfoque muestra una tendencia importante: definir el dominio en un nivel conceptual o declarativo y obtener después clases, validadores o esquemas de forma automática. La arquitectura de Open CVN adopta esta idea general de generación y trazabilidad, aunque la adapta a las fuentes disponibles en el ecosistema CVN y a las necesidades concretas del proyecto.

Otra herramienta relacionada es LinkML, un lenguaje de modelado orientado a datos estructurados que permite definir esquemas de forma declarativa y generar artefactos en distintos formatos [LinkML, 2026]. LinkML resulta especialmente interesante por su capacidad para actuar como puente entre modelos, esquemas JSON, documentación y tecnologías de datos enlazados. No obstante, para el alcance de este TFG, su adopción completa introduciría una capa adicional de complejidad que no resulta imprescindible. Por ello, se considera como referencia conceptual, pero no como dependencia central de la solución desarrollada.

2.5. Síntesis del estado del arte

El análisis realizado permite extraer varias conclusiones relevantes para el desarrollo de una representación curricular abierta. En primer lugar, los modelos relacionados con información de investigación, como CERIF, VIVO o ROH, muestran soluciones expresivas y semánticamente ricas, aunque su complejidad puede resultar desproporcionada para una herramienta abierta y ligera de gestión curricular. En segundo lugar, los formatos de serialización deben entenderse como mecanismos de intercambio y almacenamiento, no como sustitutos del modelo conceptual. En tercer lugar, UML ofrece una notación adecuada para representar el esquema conceptual de forma comprensible antes de materializarlo en modelos ejecutables o esquemas de validación.

Estas observaciones justifican la necesidad de una arquitectura propia que separe modelo conceptual, serialización y validación, preserve trazabilidad hacia las fuentes oficiales del dominio y genere una representación final más adecuada para validación, almacenamiento y

exportación. De este modo, el estado del arte no conduce a adoptar sin modificaciones una ontología completa ni a reproducir directamente una estructura de intercambio existente, sino a proponer una solución intermedia: suficientemente formal para ser validable, suficientemente trazable para conservar el significado curricular y suficientemente ligera para servir como base de herramientas abiertas.

3. Análisis del ecosistema CVN y propuesta de solución

El Capítulo 2 concluyó que una representación curricular abierta debe combinar una separación clara entre modelo conceptual, serialización y validación con una trazabilidad explícita hacia las fuentes normativas del dominio. El presente capítulo aplica esa conclusión al caso concreto de este Trabajo Fin de Grado. En primer lugar, se examina en detalle el contenido real del paquete oficial de CVN empleado como fuente del proyecto. A continuación, se identifican las limitaciones técnicas que dicho paquete introduce cuando se pretende utilizarlo directamente como modelo interno de una aplicación. Finalmente, a partir de ese análisis, se presenta la arquitectura Open CVN que se desarrolla con más detalle en los capítulos siguientes.

3.1. El paquete oficial CVN como fuente del proyecto

El Capítulo 1 ya situó a CVN como la norma española de referencia para la presentación de información curricular de investigadores, mantenida por FECYT [FECYT, 2026]. Ese capítulo, sin embargo, se limitó a introducir CVN como motivación del proyecto, sin describir el contenido técnico del paquete que FECYT distribuye para su implementación. Esa descripción es el punto de partida de este apartado.

El paquete oficial empleado como fuente de este trabajo no es un único fichero, sino un conjunto heterogéneo de artefactos que puede organizarse en cuatro capas complementarias. La primera capa es un manual de especificaciones técnicas en formato PDF, orientado a lectura humana, que define el significado funcional de cada campo curricular, su tipo de dato, su obligatoriedad y la tabla de referencia asociada cuando corresponde. La segunda capa traduce

ese mismo manual a un fichero XML estructurado, pensado para su explotación automática en lugar de su lectura directa. La tercera capa es un modelo en árbol que enlaza cada código funcional del manual con las propiedades e indicadores concretos que aparecen en un documento CVN real serializado en XML. La cuarta capa está formada por los esquemas XSD que definen la forma legal de ese XML, incluyendo un vocabulario extenso de tablas auxiliares codificadas como enumeraciones.

Junto a estas cuatro capas, el paquete incluye tres familias auxiliares que no forman parte del núcleo mínimo de codificación, pero que resultan necesarias para interpretar correctamente numerosas referencias del manual: un catálogo normalizado de entidades e instituciones, una materialización en XML de buena parte de las tablas del anexo de valores controlados junto con la codificación de subtipos, y un tesoro jerárquico multilingüe de materias y palabras clave. Sin estas familias auxiliares, una parte relevante de los campos del manual quedaría referida a catálogos externos sin forma de resolverse dentro del propio paquete.

La Tabla 3.1 resume, para cada una de estas fuentes, la información principal que aporta y el uso concreto que recibe dentro del proyecto.

Estas fuentes no son intercambiables entre sí. El manual en PDF y su versión estructurada en XML son la mejor base para entender *qué significa* cada campo, mientras que el modelo en árbol y los esquemas XSD explican *cómo se serializa* ese mismo campo dentro de un documento XML concreto. Esta distinción, introducida aquí de forma descriptiva, es la que justifica más adelante por qué la propuesta Open CVN separa explícitamente el análisis semántico del dominio de su representación técnica en XML.

Fuente	Información que aporta	Uso dentro del proyecto
Manual de especificaciones técnicas (PDF)	Semántica funcional de cada campo curricular, tipo de dato y obligatoriedad	Referencia normativa para interpretar el significado de los códigos CVN
Manual estructurado en XML	Catálogo del manual en forma parseable: código, nombre, tipo, obligatoriedad y tabla de referencia	Fuente principal para la extracción automática de metadatos funcionales
Modelo en árbol XML	Enlace entre cada código funcional y las propiedades e indicadores del XML CVN real	Trazabilidad estructural entre el dominio funcional y la serialización XML
Esquemas XSD del núcleo (documento, tipos comunes y tablas auxiliares)	Forma legal del XML final y vocabulario de tablas codificadas como enumeraciones	Generación reproducible de bindings estructurales y validación de forma
Tablas de referencia y subtipos	Materialización en XML de gran parte del anexo de valores controlados y de la codificación de subtipos	Resolución de referencias auxiliares y evaluación de enumeraciones cerradas
Catálogo de entidades	Registro normalizado de instituciones y entidades	Resolución de referencias de tipo registro
Tesaurus multilingüe	Vocabulario jerárquico de materias y palabras clave	Resolución de referencias de tipo tesaurus

Tabla 3.1: Fuentes del paquete oficial CVN analizadas y su uso en el proyecto.

3.2. El XML embebido en los documentos CVN

Además del paquete de especificación descrito en el apartado anterior, la herramienta oficial de FECYT genera, para cada currículum, un documento en formato PDF pensado principalmente para su presentación y lectura humana. Una parte relevante de estos documentos conserva, embebido dentro del propio fichero PDF, el XML CVN que dio lugar a su generación. Cuando ese XML embebido existe y resulta accesible, constituye una fuente adicional de información curricular ya estructurada, en lugar de un simple documento visual del que haya que extraer texto de forma no estructurada.

Esta circunstancia tiene una implicación práctica directa para el procesamiento automático de currículos ya existentes: un sistema que pretenda incorporar currículos previamente generados por la herramienta oficial puede intentar, en primer lugar, recuperar y validar ese XML embebido, y solo recurrir a mecanismos menos deterministas cuando dicha extracción no sea posible. Este trabajo adopta precisamente ese criterio de prioridad, cuyo desarrollo funcional completo se presenta en el Capítulo 6.

3.3. Limitaciones y dificultades técnicas detectadas

El análisis detallado del paquete oficial revela un conjunto de limitaciones técnicas que, sin restar validez a la norma CVN, condicionan de forma directa cualquier intento de utilizar sus artefactos como modelo interno de una aplicación. Estas limitaciones se agrupan en cuatro tipos.

En primer lugar, existen discrepancias puntuales entre el XML canónico del modelo en árbol y el esquema XSD que se supone que lo valida. El caso más representativo aparece en dos indicadores concretos del modelo en árbol, que incluyen un elemento hijo no declarado por dicho esquema. De los cientos de indicadores del mismo tipo presentes en el documento, únicamente dos contienen ese elemento adicional, y ambos hacen referencia al mismo valor de una tabla auxiliar de tipos de calidad. Esta discrepancia impide que unas estructuras generadas directamente desde el esquema puedan analizar por completo el XML canónico, y obliga a tratar dicho XML como evidencia de origen que debe registrarse explícitamente en lugar de forzarse a encajar en la forma legal definida por el esquema.

En segundo lugar, ciertas construcciones del propio lenguaje de esquemas no se trasladan de forma perfecta a estructuras de datos ejecutables. El mecanismo de composición *choice*, que en XSD permite declarar alternativas entre varios elementos posibles, no siempre puede imponerse como una restricción de exclusión mutua real sobre las estructuras generadas

automáticamente a partir del esquema. De forma similar, algunas cardinalidades mínimas declaradas en el esquema no quedan garantizadas por dichas estructuras, y varios atributos se generan con un tipo demasiado genérico, sin estructura interna definida. Estas limitaciones son consistentes con una idea ya avanzada en el Capítulo anterior: un esquema de validación describe la forma legal de un documento, pero no captura por completo las reglas semánticas del dominio que representa.

En tercer lugar, no todas las tablas auxiliares del paquete se comportan como enumeraciones cerradas. Algunas incluyen valores de tipo *delegado* o comportamiento abierto que impide tratarlas como un catálogo fijo sin perder información, mientras que otras sí son suficientemente estables y compactas para ese tratamiento. Determinar esta distinción exige examinar la evidencia concreta de cada tabla en lugar de asumir un criterio uniforme para todas ellas.

En cuarto lugar, algunas referencias mencionadas en el manual funcional no tienen una tabla equivalente clara dentro del resto del paquete, y las tres familias auxiliares descritas en el apartado anterior presentan una deriva histórica de empaquetado, con nombres de fichero que no siempre coinciden entre la documentación de orientación y los artefactos realmente distribuidos. Esta circunstancia obliga a resolver dichas familias mediante un mapeo consciente del paquete, en lugar de asumir convenios de nombre fijos.

En conjunto, estas cuatro limitaciones evidencian un mismo problema de fondo: la estructura XML del paquete oficial y el significado curricular que dicha estructura pretende representar no están completamente alineados entre sí. La Tabla 3.2 resume estas limitaciones junto con la respuesta de diseño que se adopta para cada una de ellas dentro de la arquitectura Open CVN.

Ninguna de estas limitaciones invalida la norma CVN ni el paquete distribuido por FECYT, que cumplen adecuadamente su función original de interoperabilidad documental. Lo que estas limitaciones muestran es que dicho paquete no puede tomarse, sin mediación adicional, como el modelo interno definitivo de una herramienta de gestión curricular. Esa mediación adicional es precisamente el objeto de la propuesta de solución que se presenta a continuación.

3.4. Propuesta de solución: la arquitectura Open CVN

Copiar directamente la estructura del XML oficial como modelo interno de una aplicación resolvería la interoperabilidad con CVN, pero trasladaría a dicha aplicación todas las limitaciones descritas en el apartado anterior, y mezclaría dos preocupaciones de naturaleza distinta: la interoperabilidad con un formato de intercambio externo y el modelado semántico del dominio curricular. Por este motivo, Open CVN no adopta el XML oficial como modelo final, sino que lo trata como una fuente formal de la que se parte mediante un proceso reproducible y explícitamente trazable.

Esta decisión no implica renunciar a la relación con CVN. Al contrario: la propuesta exige conservar, para cada elemento del modelo final, evidencia verificable de la fuente CVN de la que procede, de modo que la transformación hacia un formato propio no suponga una pérdida de significado curricular. Esta exigencia de trazabilidad se traduce en una arquitectura organizada por capas, en la que cada capa añade una interpretación adicional sobre la capa anterior sin descartar la relación con el paquete oficial. La Figura 3.1 representa esta idea de forma esquemática.

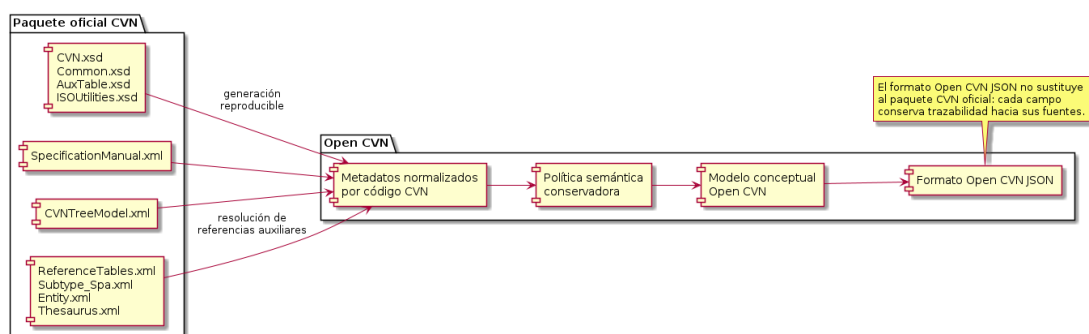


Figura 3.1: Separación entre el paquete oficial CVN y la arquitectura Open CVN.

Como muestra la Figura 3.1, el paquete oficial completo, incluidas sus familias auxiliares, se convierte en primer lugar en metadatos normalizados por código CVN, un paso que unifica en una única estructura la información dispersa entre el manual, el modelo en árbol y las fuentes auxiliares. Sobre esos metadatos normalizados actúa una política semántica conservadora, que decide de forma explícita y trazable cómo debe interpretarse cada campo: como texto, fecha, número, enumeración cerrada, catálogo abierto, referencia a una entidad, término de tesoro o referencia no resuelta, entre otros casos. El resultado de esta política alimenta un

modelo conceptual de dominio que ya no depende de la estructura concreta del XML CVN, y que constituye la base a partir de la cual se deriva finalmente el formato Open CVN JSON descrito en el Capítulo 6.

Este modelo conceptual se representa mediante diagramas de clases UML, tal y como se anticipó en el Capítulo 2. La Figura 3.2 muestra una vista compacta de dicho modelo, en la que un documento Open CVN se organiza en metadatos, un bloque de currículum y un espacio de extensión, y en la que ese bloque de currículum se subdivide en áreas conceptuales del dominio académico e investigador. Los nombres de clase de esta vista coinciden deliberadamente con las claves técnicas del formato Open CVN JSON, de modo que la figura sirva también como puente directo hacia la estructura raíz que se detalla en el Capítulo 6.

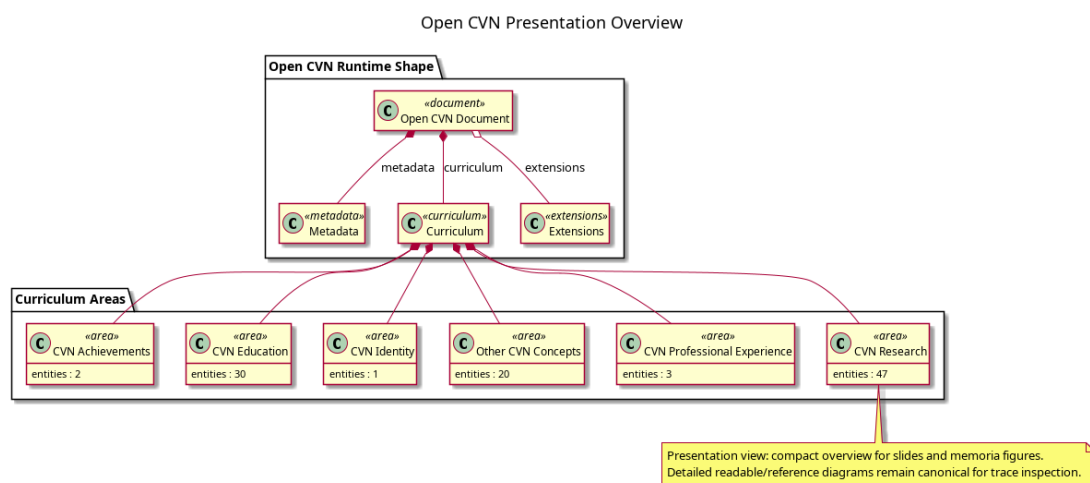


Figura 3.2: Vista UML compacta del esquema conceptual de Open CVN, con el número de conceptos identificados por área curricular.

Sobre esta base conceptual y de trazabilidad se construyen finalmente los mecanismos que hacen operativa la propuesta: un formato Open CVN JSON validable y extensible, y un conjunto de utilidades de análisis, validación e importación que, junto con una herramienta local de gestión, demuestran que la representación definida puede emplearse en flujos reales de trabajo curricular. Esta herramienta local cumple una función demostrativa del enfoque, mientras que la aportación principal de la propuesta reside en la arquitectura de representación, normalización y trazabilidad descrita en este apartado.

3.5. Alcance y exclusiones

El análisis anterior permite delimitar con precisión qué garantías ofrece realmente la arquitectura Open CVN frente al ecosistema CVN del que parte.

Se automatizan con garantías reproducibles la generación de la capa estructural desde los esquemas oficiales, la normalización de metadatos por código CVN a partir del manual y del modelo en árbol, la resolución de referencias auxiliares cuando existe evidencia suficiente en el paquete, la aplicación de la política semántica conservadora y la generación del modelo conceptual y del formato Open CVN JSON a partir de dicho modelo.

Se mantienen como mapeo parcial, y así deben entenderse, la interpretación de tablas auxiliares sin evidencia suficiente para una enumeración cerrada, que permanecen documentadas como abiertas o pendientes de revisión, y la incorporación de currículos CVN ya existentes en formato XML, que se resuelve mediante un mapeo semántico de los elementos reconocidos del documento en lugar de una conversión completa y garantizada de cualquier XML CVN posible.

Quedan documentadas explícitamente como limitaciones o trabajo futuro las referencias del manual sin tabla equivalente clara en el resto del paquete, la deriva de empaquetado entre familias auxiliares, y la ausencia de una garantía formal de que el modelo conceptual capture la totalidad de las reglas de negocio implícitas en el manual funcional. Ninguna de estas exclusiones se oculta: todas ellas se retoman de forma explícita en la discusión de garantías del Capítulo 7.

En conjunto, el análisis realizado en este capítulo justifica por qué Open CVN no se limita a reproducir la estructura XML de CVN, sino que introduce una arquitectura propia, trazable y validable a partir de ella. El Capítulo siguiente describe la metodología de trabajo seguida y detalla la arquitectura general por capas que materializa esta propuesta.

4. Metodología, herramientas y arquitectura general

El Capítulo 3 analizó el contenido real del paquete oficial CVN, identificó sus limitaciones técnicas y presentó, a un nivel todavía conceptual, la arquitectura por capas que Open CVN propone frente a esas limitaciones. El presente capítulo desciende un nivel de concreción: describe el proceso de trabajo seguido durante el desarrollo, justifica la selección de herramientas empleadas y detalla la arquitectura general del sistema, incluida la organización del repositorio que la materializa. Los detalles de implementación de cada capa se reservan para el Capítulo 5.

4.1. Metodología de trabajo

El desarrollo de este Trabajo Fin de Grado no siguió un proceso de ingeniería del software orientado a requisitos de usuario o iteraciones de producto, sino un proceso adaptado a un trabajo de Computación centrado en el análisis de formatos, la generación reproducible de artefactos y la verificación técnica de un sistema. Este proceso puede describirse en cinco fases sucesivas.

La primera fase consistió en un análisis inicial del dominio curricular y de los formatos de representación disponibles, cuyo resultado se recoge en el Capítulo 2. La segunda fase exploró en detalle los artefactos oficiales del paquete CVN, identificando su estructura y sus limitaciones técnicas, tal y como se presentó en el Capítulo 3. La tercera fase, desarrollada en el presente capítulo, diseñó de forma incremental una arquitectura por capas capaz de absorber esas limitaciones sin perder trazabilidad hacia las fuentes oficiales. La cuarta fase implementó dicha arquitectura de forma reproducible, generando de manera automática los

artefactos estructurales, los metadatos normalizados, los modelos de dominio y el formato Open CVN JSON, como se describe en el Capítulo 5. La quinta fase verificó el sistema mediante pruebas automatizadas y flujos completos de extremo a extremo, cuyos resultados se discuten en el Capítulo 7.

Estas cinco fases no se ejecutaron de forma estrictamente secuencial. Cada limitación detectada durante la exploración del paquete oficial obligó, en la práctica, a revisar decisiones de diseño ya tomadas en fases anteriores, y cada nueva capa incorporada exigió documentar explícitamente las decisiones y las limitaciones que introducía. Esta documentación continua de decisiones y limitaciones técnicas, más que una fase aislada, constituye un criterio transversal de todo el proceso: ninguna limitación detectada se resolvió ocultándola, sino registrándola y decidiendo de forma explícita cómo debía tratarse en la capa correspondiente.

4.2. Herramientas empleadas

La selección de herramientas responde a un criterio común: cada herramienta debía permitir generar o validar artefactos de forma reproducible a partir de fuentes formales, sin introducir dependencias que condicionaran en exceso la portabilidad del sistema.

El proyecto se implementa en Python, cuya combinación de tipado gradual y ecosistema maduro de procesamiento de XML y JSON resulta adecuada para implementar las sucesivas capas de generación y transformación descritas en este documento. La gestión del entorno y de las dependencias se realiza con *uv*, un gestor de paquetes y proyectos Python que permite fijar de forma determinista tanto la versión del intérprete como las dependencias empleadas [Astral, 2026], un requisito necesario para que la generación de artefactos sea reproducible entre distintos entornos de ejecución.

La generación de la capa estructural a partir de los esquemas XSD oficiales se realiza con *xsddata*, una biblioteca de generación de enlaces de datos que permite obtener clases Python tipadas directamente desde esquemas XML sin necesidad de traducirlos manualmente [xsddata, 2026]. Esta herramienta se complementa con su extensión para Pydantic, que emite las clases generadas ya como modelos Pydantic en lugar de estructuras de datos genéricas.

Pydantic actúa como base de los modelos de dominio, del formato Open CVN JSON y de buena parte de la validación en tiempo de ejecución del sistema. Como ya se introdujo en el Capítulo 2, Pydantic combina anotaciones de tipo, validación de datos y generación de JSON Schema en un mismo mecanismo [Pydantic, 2026], lo que permite mantener alineados el código Python, el contrato de validación y la documentación estructural del formato. Ese contrato de validación se expresa como JSON Schema, cuya versión Draft 2020-12 es la

empleada para validar de forma declarativa el formato Open CVN JSON antes de aplicar las comprobaciones adicionales en tiempo de ejecución [JSON Schema, 2026].

El almacenamiento local de currículos, versiones y diagnósticos se apoya en SQLite, un motor de bases de datos embebido que no requiere un servidor independiente [SQLite, 2026]. Esta elección evita introducir una dependencia de infraestructura externa y mantiene la herramienta local autocontenida. La exportación a documentos LaTeX se implementa con Jinja, un motor de plantillas que separa la lógica de presentación de los datos ya validados [Pallets Projects, 2026], de modo que la generación de un documento LaTeX no exige mezclar código Python con marcado de presentación.

La verificación automatizada del sistema se realiza con *pytest* como marco de pruebas, junto con su extensión *pytest-xdist* para la ejecución en paralelo de la batería de pruebas [pytest, 2026], lo que permite mantener tiempos de ejecución razonables a pesar del volumen de comprobaciones necesarias para cubrir las distintas capas del sistema. El control de versiones se realiza con Git, y cada contribución se verifica de forma automática mediante GitHub Actions, un servicio de integración continua que ejecuta la batería de pruebas ante cualquier cambio propuesto sobre el repositorio [GitHub, 2026].

La Tabla 4.1 resume estas herramientas junto con su papel principal y la justificación de su uso dentro del proyecto.

Herramienta	Papel en el proyecto	Justificación
Python + <i>uv</i>	Lenguaje de implementación y gestión reproducible del entorno y las dependencias	Tipado gradual, ecosistema maduro para XML/JSON y fijación determinista del entorno de ejecución
<i>xsdata</i> + extensión Pydantic	Generación de bindings estructurales desde los esquemas XSD oficiales	Evita traducir manualmente esquemas complejos y mantiene los bindings alineados con la fuente oficial
Pydantic	Modelos de dominio, formato Open CVN JSON y validación en tiempo de ejecución	Combina tipado, validación y generación de JSON Schema en un mismo mecanismo
JSON Schema (Draft 2020-12)	Validación estructural declarativa del formato Open CVN JSON	Contrato de validación independiente del lenguaje de implementación
SQLite	Almacenamiento local de currículos, versiones y diagnósticos	Motor embebido que evita una dependencia de infraestructura externa
Jinja + motor LaTeX	Exportación de currículos almacenados a documentos LaTeX y PDF	Separa la plantilla de presentación de los datos ya validados
<i>pytest</i> + <i>pytest-xdist</i>	Verificación automatizada y ejecución paralela de la batería de pruebas	Mantiene tiempos de ejecución razonables sobre una batería de pruebas extensa
Git + GitHub Actions	Control de versiones e integración continua	Verifica automáticamente cada contribución antes de integrarse

Tabla 4.1: Herramientas empleadas en el proyecto y justificación de su uso.

4.3. Arquitectura general por capas

La arquitectura general del sistema traduce en artefactos concretos la separación conceptual presentada en el Capítulo 3 entre el paquete oficial CVN y el modelo Open CVN derivado de él. La Figura 4.1 representa esta arquitectura agrupada en cuatro grandes bloques, cada uno de los cuales corresponde a una parte concreta del repositorio del proyecto.

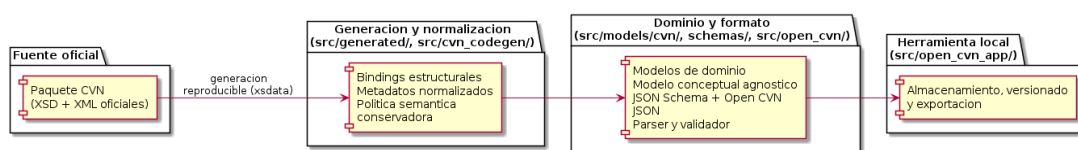


Figura 4.1: Arquitectura general de Open CVN organizada por capas, con el módulo del repositorio responsable de cada bloque.

Cada uno de estos cuatro bloques agrupa, a su vez, una secuencia de etapas más concretas. Los apartados siguientes describen esa secuencia bloque a bloque, indicando para cada etapa qué recibe, qué produce y qué módulo del repositorio la implementa. El detalle interno de cómo se implementa cada etapa, incluidos los algoritmos y los resultados numéricos de su ejecución, se reserva para el Capítulo 5; este apartado se limita a establecer el mapa general de responsabilidades entre etapas.

4.3.1. Fuente oficial CVN

El primer bloque no realiza ningún procesamiento propio: corresponde al paquete oficial CVN ya descrito en el Capítulo 3, y actúa como frontera externa y única fuente formal de la que parte el resto de la arquitectura. Ninguna etapa posterior incorpora información curricular que no proceda, directa o indirectamente, de este bloque.

4.3.2. Generación estructural y normalización

El segundo bloque transforma el paquete oficial en metadatos internos normalizados y semánticamente interpretados, en cuatro etapas sucesivas:

- **Generación estructural.** A partir de los esquemas XSD oficiales se generan de forma reproducible bindings Pydantic que reflejan fielmente la estructura del XML de CVN.

Esta etapa produce los bindings estructurales almacenados en `src/generated/` y no incorpora todavía ninguna interpretación semántica de los campos.

- **Normalización.** A partir del manual funcional en XML y del modelo en árbol, esta etapa unifica en una única estructura, indexada por código CVN, la información hasta entonces dispersa entre ambas fuentes. Recibe el paquete oficial y produce metadatos normalizados por código.
- **Resolución de referencias auxiliares.** Sobre los metadatos normalizados, esta etapa resuelve las referencias del manual hacia las familias auxiliares del paquete descritas en el Capítulo 3, añadiendo a cada metadato la evidencia de resolución disponible.
- **Política semántica.** A partir de los metadatos ya resueltos, esta etapa decide de forma explícita y trazable la forma semántica de cada campo (texto, fecha, número, enumeración cerrada, catálogo abierto, referencia a entidad, término de tesauro o referencia no resuelta, entre otras), produciendo una política semántica completa del dominio.

Las cuatro etapas de este bloque se implementan como lógica mantenida a mano en `src/cvn_codegen/`, con la única excepción de los bindings generados por la primera etapa, que se almacenan de forma separada en `src/generated/`.

4.3.3. Dominio, modelo conceptual y formato

El tercer bloque construye, a partir de la política semántica, las representaciones que ya no dependen de la estructura concreta del XML de CVN:

- **Modelos de dominio.** A partir de la política semántica se generan modelos de dominio Pydantic, almacenados en `src/models/cvn/` junto con un conjunto reducido de componentes de dominio compartidos mantenidos a mano.
- **Modelo conceptual agnóstico.** A partir de los modelos de dominio y de las decisiones de normalización y política semántica que los originaron, esta etapa construye el inventario conceptual presentado en el Capítulo 3, ya independiente de la serialización XML.
- **JSON Schema y formato Open CVN JSON.** A partir del modelo conceptual se genera el artefacto JSON Schema, almacenado en `schemas/`, que define de forma declarativa la estructura válida del formato Open CVN JSON.
- **Parser y validador.** Sobre el JSON Schema y los modelos de dominio se apoya el contrato público de análisis y validación, almacenado en `src/open_cvn/`, que expone

la importación de Open CVN JSON, CVN XML y CVN PDF junto con la validación en tiempo de ejecución.

Las tres primeras etapas de este bloque se mantienen en `src/cvn_codegen/` y `src/models/cvn/`; la última corresponde ya al módulo `src/open_cvn/`, que constituye el límite público del sistema hacia cualquier código externo que necesite leer o validar currículos.

4.3.4. Herramienta local

El cuarto bloque no añade una etapa de transformación de datos, sino que consume el contrato público de análisis y validación para ofrecer un uso operativo completo: inicialización de almacenamiento, importación y exportación, gestión de un currículo maestro y de versiones derivadas, selección de secciones y entradas, y exportación a LaTeX y PDF. Todo este bloque se implementa en `src/open_cvn_app/`.

La Tabla 4.2 consolida, para cada módulo del repositorio mencionado en los apartados anteriores, la responsabilidad que le corresponde dentro de la arquitectura Open CVN.

Módulo	Responsabilidad
<code>src/generated/</code>	Bindings estructurales Pydantic generados automáticamente desde los esquemas XSD oficiales; no se edita manualmente
<code>src/cvn_codegen/</code>	Lógica mantenida a mano: normalización, resolución de referencias auxiliares, política semántica, generación de modelos de dominio, extracción del modelo conceptual y generación de diagramas y JSON Schema
<code>src/models/cvn/</code>	Componentes de dominio compartidos mantenidos a mano y modelos de dominio generados automáticamente a partir de la política semántica
<code>schemas/</code>	Artefacto JSON Schema generado del formato Open CVN
<code>src/open_cvn/</code>	Contrato público de análisis y validación: importación de Open CVN JSON, CVN XML y CVN PDF, y validación en tiempo de ejecución
<code>src/open_cvn_app/</code>	Herramienta local: interfaz de línea de órdenes, almacenamiento SQLite, versionado, edición de selección y exportación a LaTeX y PDF

Tabla 4.2: Módulos del repositorio y responsabilidades dentro de la arquitectura Open CVN.

4.4. Contratos entre capas y razonamiento arquitectónico

La separación entre módulos descrita en el apartado anterior no es únicamente organizativa: cada frontera entre módulos corresponde a un contrato explícito que determina qué información puede cruzar de una capa a otra. Generar un modelo de dominio final directamente desde los esquemas XSD oficiales, sin capas intermedias, mezclaría dos preocupaciones de naturaleza distinta: la interoperabilidad estructural con el XML de CVN y el modelado semántico del dominio curricular. Por ello, los bindings estructurales generados automáticamente se tratan como una capa de fidelidad hacia el XML oficial, y cualquier limpieza semántica o ergonómica se realiza en capas posteriores, en lugar de modificarse a mano sobre el código generado.

Esta decisión tiene una consecuencia práctica directa sobre la reproducibilidad del sistema: dado que la capa estructural puede regenerarse en cualquier momento a partir de los esquemas oficiales sin perder trabajo manual, las limitaciones que dicha capa presenta, ya descritas en el Capítulo 3, quedan documentadas en lugar de enmascarse mediante ediciones manuales frágiles. De forma análoga, los modelos de dominio y el modelo conceptual no vuelven a inspeccionar el XML o los esquemas XSD originales, sino que consumen exclusivamente las decisiones ya tomadas por la normalización y por la política semántica, evitando así que la semántica del dominio se derive de forma inconsistente en distintos puntos del sistema. El contrato público de análisis y validación, a su vez, expone únicamente el formato Open CVN JSON y sus resultados de validación, sin obligar a quien lo utiliza a conocer los detalles internos de generación de las capas anteriores.

En conjunto, esta arquitectura por capas materializa de forma concreta la propuesta presentada en el Capítulo 3: conserva trazabilidad hacia el paquete oficial CVN en cada etapa, mantiene separadas la interoperabilidad estructural y la semántica de dominio, y permite verificar de forma independiente cada capa mediante pruebas automatizadas. El Capítulo 5 describe en detalle cómo se implementa cada una de estas capas, desde la generación estructural hasta la extracción del modelo conceptual y la generación de diagramas y JSON Schema.

Referencia bibliográfica

- [Astral, 2026] Astral (2026). uv: An extremely fast python package and project manager. <https://docs.astral.sh/uv/>. Accedido: 2026-07-23.
- [BESSER, 2026] BESSER (2026). Pydantic generator - BESSER documentation. <https://besser.readthedocs.io/en/latest/generators/pydantic.html>. Accedido: 2026-02-25.
- [Deusto HERCULES, 2026a] Deusto HERCULES (2026a). CVN. <https://github.com/deustohercules/CVN>. Accedido: 2026-02-16.
- [Deusto HERCULES, 2026b] Deusto HERCULES (2026b). ROH-1. <https://github.com/deustohercules/ROH-1>. Accedido: 2026-02-16.
- [euroCRIS, 2026] euroCRIS (2026). Main features of CERIF. <https://eurocris.org/services/main-features-cerif>. Accedido: 2026-02-16.
- [FECYT, 2026] FECYT (2026). Norma CVN. <https://cvn.fecyt.es/>. Accedido: 2026-02-16.
- [GitHub, 2026] GitHub (2026). GitHub Actions documentation. <https://docs.github.com/en/actions>. Accedido: 2026-07-23.
- [JSON Schema, 2026] JSON Schema (2026). What is JSON Schema? <https://json-schema.org/overview/what-is-jsonschema>. Accedido: 2026-07-22.
- [JSON.org, 2026] JSON.org (2026). Introduccion a JSON. <https://json.org/json-es.html>. Accedido: 2026-02-19.

-
- [LinkML, 2026] LinkML (2026). Linked data modeling language (LinkML). <https://linkml.io/>. Accedido: 2026-02-23.
- [Object Management Group, 2026a] Object Management Group (2026a). Object constraint language (OCL). <https://www.omg.org/spec/OCL/>. Accedido: 2026-02-25.
- [Object Management Group, 2026b] Object Management Group (2026b). Unified modeling language (UML). <https://www.omg.org/uml/>. Accedido: 2026-02-25.
- [ORCID, 2026] ORCID (2026). About ORCID. <https://info.orcid.org/what-is-orcid/>. Accedido: 2026-07-22.
- [Pallets Projects, 2026] Pallets Projects (2026). Jinja documentation. <https://jinja.palletsprojects.com/>. Accedido: 2026-07-23.
- [Proyecto HERCULES, 2026] Proyecto HERCULES (2026). Research ontology for hercules (ROH). <https://deustohercules.github.io/roh/roh/index.html>. Accedido: 2026-02-16.
- [Pydantic, 2026] Pydantic (2026). Pydantic documentation. <https://docs.pydantic.dev/latest/>. Accedido: 2026-07-22.
- [pytest, 2026] pytest (2026). pytest documentation. <https://docs.pytest.org/en/stable/>. Accedido: 2026-07-23.
- [RO-Crate Community, 2024] RO-Crate Community (2024). Research object crate (RO-Crate). <https://www.researchobject.org/ro-crate/>. Accedido: 2026-07-22.
- [SQLite, 2026] SQLite (2026). SQLite home page. <https://www.sqlite.org/index.html>. Accedido: 2026-07-23.
- [Universidad de Murcia, 2026] Universidad de Murcia (2026). Ontologias - proyecto HERCULES. <https://www.um.es/web/hercules/ontologias>. Accedido: 2026-02-16.
- [VIVO Project, 2026] VIVO Project (2026). VIVO: Connect, share, discover. <https://www.vivoweb.org/>. Accedido: 2026-02-16.
- [World Wide Web Consortium, 2008] World Wide Web Consortium (2008). Extensible markup language (XML) 1.0 (fifth edition). <https://www.w3.org/TR/xml/>. W3C Recommendation, accedido: 2026-07-22.
-

[World Wide Web Consortium, 2012] World Wide Web Consortium (2012). W3C XML Schema Definition Language (XSD) 1.1 part 1: Structures. <https://www.w3.org/TR/xmlschema11-1/>. W3C Recommendation, accedido: 2026-07-22.

[World Wide Web Consortium, 2020] World Wide Web Consortium (2020). JSON-LD 1.1: A JSON-based serialization for linked data. <https://www.w3.org/TR/json-ld11/>. W3C Recommendation, accedido: 2026-07-22.

[xsdata, 2026] xsdata (2026). xsdata documentation. <https://xsdata.readthedocs.io/en/latest/>. Accedido: 2026-07-23.

[YAML.org, 2026] YAML.org (2026). YAML ain't markup language. <https://yaml.org/>. Accedido: 2026-02-23.